Command Reference & Multi-Pipeline Operational Guide

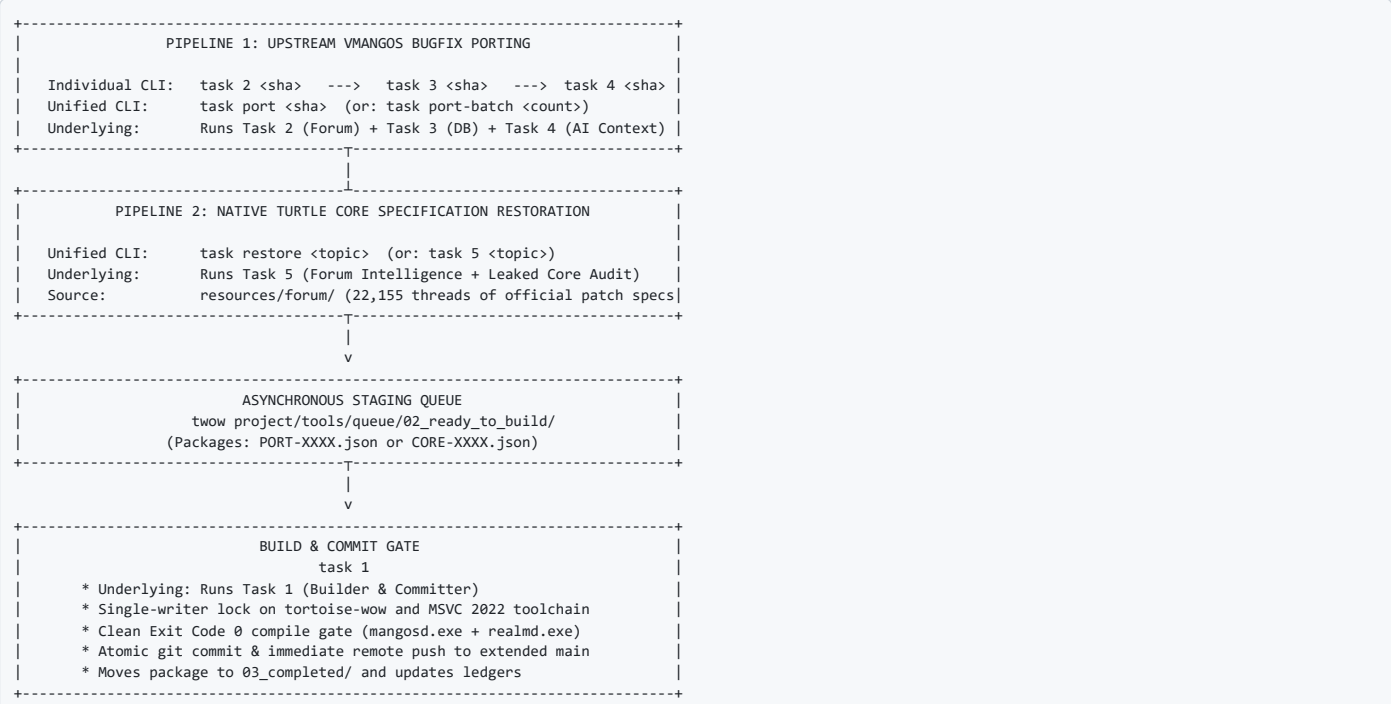
This document is the master CLI operational reference for **Tortoise-WoW Extended** (`twow project/tortoise-wow`), documenting both the **Upstream Porting Pipeline** (VMaNGOS bugfix backporting) and the **Native Core Restoration Pipeline** (Turtle-WoW leaked core specification restoration).

> **TIP**
> **RUNNING FROM ANY DIRECTORY OR EMPTY CLI TERMINAL** You do **not** need to `cd` into the project folder. You can run all commands directly from any empty terminal or PowerShell prompt using the full path: ```powershell & "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" <command> [arguments] ``` Or from Windows cmd / bash / shortcuts: ```cmd powershell.exe -ExecutionPolicy Bypass -File "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" <command> [arguments] ```

---

1. Pipeline Architecture Overview

The system operates across two distinct discovery and adaptation channels feeding into a single unified build and release gate:



---

2. Master Command Reference Table

All commands can be run directly from any terminal location using the full script path `& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1"` :

Command	Full Invocation (From Any Directory)	Pipeline	Underlying Tasks Executed	Role & Functionality	Output Location
`task status`	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" status`	**System**	**None** *(Status Query)*	Displays live metrics: pending candidates, ready packages, staging patches, completed count, rejected count, and git HEAD.	Console Output
`task port <sha>`	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port <sha>`	**AI Pipeline**	**AI Context Assembler (Tasks 2, 3, 4)**	**Unified AI Upstream Port**: Runs AI Semantic Context Assembler, mines forum lore, checks Turtle code context, and stages as `READY_FOR_BUILD` or `AWAITING_AI_ADAPTATION` (no blind regex rejections).	`02_ready_to_build/`
`task port <sha> -AutoBuild`	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port <sha> -AutoBuild`	**AI Pipeline**	**AI Assembler, then Task 1**	**Autonomous AI Port & Build**: Runs AI porting checks, and if cleanly applicable, immediately compiles via MSVC, commits, and pushes!	Remote `extended main`
`task port-batch <N>`	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port-batch <N>`	**AI Pipeline**	**AI Assembler (loops N times)**	**Batch AI Port**: Automatically pulls next N candidates from queue, generates AI dossiers, and stages all viable/adaptable ones.	`02_ready_to_build/`
`task port-batch <N> -Tier <T>`	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port-batch <N> -Tier <T>`	**AI Pipeline**	**AI Assembler (loops N times)**	**Tier-Filtered Batch**: Pulls next N candidates filtered strictly by severity tier (1 to 5) with AI dossiers.	`02_ready_to_build/`
`task port-batch <N> -AutoBuild`	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port-batch <N> -AutoBuild`	**AI Pipeline**	**AI Assembler, then Task 1** *(per viable commit)*	**Full Hands-Off Batch**: Audits N candidates with AI, stages them, and compiles & pushes clean ones sequentially via Task 1.	Remote `extended main`
`task restore <topic>` *(alias: `task 5 <topic>`)*	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" restore "<topic>"` *(or: `& "..." 5 "<topic>"`)*	**AI Pipeline**	**Task 5 (Forum & Code AI)**	**Unified Core Restorer**: Searches forum archive for official staff posts, audits `tortoise-wow` code/DB for missing features, reports discrepancies, and logs to `docs/CORE_RESTORATION_LEDGER.md` (zero double-checking).	Console Report
`task restore-batch <N>`	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" restore-batch <N>`	**AI Pipeline**	**Task 5 (Batch Loops)**	**Batch Core Restorer**: Sequentially audits next N un-audited Turtle patch topics from the curated priority queue, skipping already-verified topics.	`02_ready_to_build/`
`task restore <topic> -StageTemplate` *(alias: `task 5 ...`)*	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" restore "<topic>" -StageTemplate` *(or: `& "..." 5 "<topic>" -StageTemplate`)*	**AI Pipeline**	**Task 5 (Forum & Code AI)**	**Core Restorer with Manifest**: Audits topic and templates a ready-to-fill package manifest `CORE-XXXX.json` .	`02_ready_to_build/`
`task restore <topic> -AutoBuild` *(alias: `task 5 ...`)*	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" restore "<topic>" -AutoBuild` *(or: `& "..." 5 "<topic>" -AutoBuild`)*	**AI Pipeline**	**Task 5, then Task 1**	**Restore & Build**: Audits topic, stages package, and runs Task 1 if code patch is staged.	Remote `extended main`
`task 6 <id/query>` *(alias: `task db-viewer`)*	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" 6 <id_or_name>` *(or: `& "..." 6 changelog`)*	**Online DB**	**Agent 6 (Web Oracle)**	**Online DB Oracle**: Queries official Turtle DB Viewer (`https://xian55.github.io/tortoise-db-viewer/`), tracks live CDN changelogs, and checks 3D models/tooltips.	Web / Console
`task scalp <tbl> <query>` *(alias: `task extract`)*	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" scalp <tbl> <id/name> [-Diff] [-Export]` *(or: `& "..." extract item 19019 -Diff`)*	**Database**	**Agent 3 (Scalper Engine)**	**Database Scalper & Diff Engine**: Scalps entities from 143MB Brotalnia DB & Turtle base SQL, maps columns, shows side-by-side diffs, strips progressive columns, and generates sanitized SQL migrations.	Console / `staging_sql/`
`task 3 [table/sha]`	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" 3 <table_or_sha>`	**Database**	**Agent 3 (Sentinel & Scalper)**	**Database Sentinel & Scalper**: If given a table/entity (`task 3 item 19019`), scalps entity; if given a migration or SHA, audits migrations against Turtle schema.	`staging_sql/`
`task 1`	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" 1`	**Builder**	**Task 1**	**Single-Writer Builder**: Compiles via MSVC 2022 Release, verifies 0 errors, commits, pushes to `extended main` , and archives packages.	Remote `extended main`
`task 2 <sha/topic>`	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" 2 <sha>`	**Pipeline 1**	**Task 2**	**Forum Scout**: Deep manual forum investigation of a single VMaNGOS commit.	`01_candidates/`
`task ai-audit <sha>` *(alias: `task 4 <sha>`)*	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" ai-audit <sha>` *(or: `& "..." 4 <sha>`)*	**AI Pipeline**	**AI Context Assembler (Task 4)**	**AI Semantic Context Assembler**: Generates full AI dossier with target files, line context, and forum intelligence.	`tools/queue/ai_dossiers/`
`task pdf`	`& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" pdf`	**Documentation**	**PDF Generator**	Regenerates `docs/COMMAND_REFERENCE.html` and `docs/COMMAND_REFERENCE.pdf` instantly.	`docs/COMMAND_REFERENCE.pdf`

3. Critical Concurrency FAQ & AutoBuild Rules

Question: Can I run `task restore <topic> -AutoBuild` and `task port <sha> -AutoBuild` concurrently?

NO. You must **NOT** execute `-AutoBuild` on two commands at the same time in separate terminals.

- **Why?**: `-AutoBuild` invokes Agent 1, which requires an **exclusive Single-Writer Lock** on `tortoise-wow`, the MSVC 2022 compiler toolchain (`ninja` / `cl.exe`), and the Git repository index (`.git/index.lock`). Running two builds simultaneously causes compiler file lock collisions and corrupted Git staging.
- **Safe Concurrent Pattern (Zero Contention)**:

Run both tasks **WITHOUT** `-AutoBuild` concurrently:

```powershell
# Terminal A (Audits and stages VMaNGOS candidate)
& "...\tools\task.ps1" port 448df9ba0

# Terminal B (Audits and stages native Turtle restoration)
& "...\tools\task.ps1" restore "Moonfury" -StageTemplate
```

Both commands operate strictly in **read-only / staging mode**, writing separate JSON packages (`PORT-0001.json` and `CORE-0001.json`) to `tools/queue/02_ready_to_build/`.

Once staged, run `task 1` once to compile, commit, and push both packages sequentially:

```powershell
& "...\tools\task.ps1" 1
```

---

4. Critical FAQ: Will `task port` Also Run `task 1` When It Ends?

By Default: NO (Runs Tasks 2, 3, and 4 Only)

If you run:

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port 84f1bbccd
```

It runs **only Tasks 2, 3, and 4 (the read-only staging pipeline)**:

1. **Task 4 probe**: Probes viability and C++ invariants.
2. **Task 2 scout**: Checks forum archive for Turtle conflicts.
3. **Task 3 sentinel**: Checks SQL schema compatibility.
4. Tests patch applicability (`git apply --check`).
5. If viable, packages it into `tools/queue/02_ready_to_build/PORT-XXXX.json`.
6. If context diverged, generates AI dossier in `tools/queue/ai_dossiers/84f1bbccd.md` and stages as `AWAITING_AI_ADAPTATION`.
7. **It stops there.** It does NOT touch the working tree, does NOT compile, and does NOT commit. This gives you full manual oversight to inspect the staged packages in `02_ready_to_build/` before building.

With `-AutoBuild`: YES (Runs Tasks 2, 3, 4, then Task 1)

If you add the `-AutoBuild` switch:

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port 84f1bbccd -AutoBuild
```

or across a batch:

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port-batch 10 -AutoBuild
```

It will:

1. Complete Tasks 2, 3, and 4 (staging checks).
2. As soon as packages are staged in `02_ready_to_build/`, it **automatically triggers Agent 1 (`task 1`)**.
3. Agent 1 applies each patch, verifies compatibility invariants, compiles MSVC Release binaries with 0 errors, commits, and pushes immediately to `extended main`!

---

4. Staging Directory Map & Lifecycle

```
twow project/tools/queue/
|-- 01_candidates/       [Active workspace for Agent 2 manual triage]
|-- 02_ready_to_build/   [Assembled packages awaiting task 1 compile gate]
|-- staging_patches/     [Unified .patch diff files created by Agent 4 or task port]
|-- staging_sql/         [Sanitized SQL migration files created by Agent 3]
|-- 03_completed/        [Permanent archive of successfully built and pushed commits]
|   |-- candidates/      [Archived triage JSON files for completed commits]
|   \-- patches/         [Archived applied .patch diffs]
\-- 04_rejected/         [Permanent archive of declined/incompatible candidates with rationale]
```

Staging Rules:

1. **Never commit staging files into git**: Staging directories exist strictly locally in `tools/queue/` and are ignored by git.

2. **Sequential Queue Drain**: Agent 1 always processes `02_ready_to_build/` in chronological/ID order (`PORT-0001` , `PORT-0002` , etc. or `CORE-0001`).

3. **Pristine State**: When `02_ready_to_build/` is empty, the repository is 100% up to date with the remote.

---

5. Daily Usage Scenarios & Examples (Executable From Any Terminal)

Scenario A: Processing the Next Batch of Crucial Bugfixes (Hands-Off)

You want to evaluate the next 20 unported commits from the roadmap and automatically build whatever is viable (Tasks 2, 3, 4 -> Task 1):

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port-batch 20 -AutoBuild
```

Scenario B: Processing Candidates with Review (Staging First)

You want to evaluate 10 candidates from Tier 3 (Combat & Formulas), inspect them first, and build later:

```
# Step 1: Stage viable packages (runs Tasks 2, 3, 4)
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port-batch 10 -Tier 3

# Step 2: Check what was staged vs. what was rejected
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" status

# Step 3: When satisfied, build, commit, and push all staged packages (runs Task 1)
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" 1
```

Scenario C: Restoring a Missing Turtle-WoW Feature from Forum Notes

You notice that a custom Turtle ability (e.g. *Holy Strike* or *Rocket Jump*) is missing from the leaked core:

```
# Step 1: Audit the forum changelogs and codebase (runs Task 5)
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" restore "Holy Strike"

# Step 2: Template a restoration package (runs Task 5 with -StageTemplate)
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" restore "Holy Strike" -StageTemplate

# Step 3: Implement the C++ logic in staging_patches/CORE-0001-holy_strike.patch
# Step 4: Build and push (runs Task 1)
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" 1
```

Scenario D: Scalping and Diffing an Entity Across Classical Databases

You need to inspect an item, creature, spell, or quest to see how it was configured in classic vanilla 1.12 vs. how Turtle-WoW modifies it:

```
# Scalp item by ID and show field-by-field diff (strips patch column)
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" scalp item 19019 -Diff

# Scalp creature by name
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" scalp creature "Onyxia" -Diff

# Scalp spell and open Online DB Viewer tooltip in browser
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" scalp spell 20925 -OpenViewer
```

Scenario E: Generating a Sanitized Database Migration Patch

You want to backport or update an entity from the reference database into Turtle-WoW without risking progressive column contamination:

```
# Auto-generate sanitized REPLACE INTO SQL in tools/queue/staging_sql/
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" scalp item 19019 -ExportSql
```

---

6. Goal-Oriented Decision Matrix: Which Command Should I Run?

Use this decision table to immediately identify which command to run based on your objective:

Your Exact Goal	Recommended Command	Mode	Next Step
Check overall pipeline health & queue counts	`task status`	Read-only	Review counts in terminal
Backport 1 specific VMaNGOS bugfix (with manual review)	`task port <sha>`	Staging	Inspect package in `02_ready_to_build/`, then run `task 1`
Backport 1 specific VMaNGOS bugfix (fully automated)	`task port <sha> -AutoBuild`	End-to-End	Build runs automatically; pushes to GitHub if clean
Backport the next N crucial fixes hands-free	`task port-batch <N> -AutoBuild`	End-to-End	Automated queue loop; compiles and commits clean fixes
Backport the next N fixes but review before building	`task port-batch <N>`	Staging	Review staged packages, then trigger `task 1`
Backport only critical crash/memory leak fixes (Tier 1)	`task port-batch <N> -Tier 1`	Staging	Stages only Tier 1 candidates; compile with `task 1`
Deep-dive into a diverged commit & see surrounding C++	`task ai-audit <sha>`	AI Dossier	Review `tools/queue/ai_dossiers/<sha>.md`
Search 22k forum threads for developer hotfixes/lore	`task 2 "<keyword>"`	Research	View ranked matching threads in console
Compare an item/NPC/spell against historical vanilla baseline	`task scalp <type> <id/name> -Diff`	Scalper	View field-by-field diff table in console
Export a clean SQL migration for an entity without `patch` column	`task scalp <type> <id> -ExportSql`	Generator	Staged in `tools/queue/staging_sql/`, move to `sql/database_updates/`
Verify live 1.18.1 client tooltips, stats, or 3D models online	`task 6 <id>` *(or `-OpenBrowser`)*	Oracle	Inspect official Turtle database viewer
Check recent official Turtle database changes/spawns	`task 6 changelog`	Oracle	View latest live CDN commits & additions
Audit a custom Turtle feature against leaked core	`task restore "<topic>"`	Parity Audit	Instant cache check (<0.05s) or scans codebase
Create a restoration package for a missing Turtle feature	`task restore "<topic> -StageTemplate"`	Restorer	Generates `CORE-XXXX.json`, write C++ patch, run `task 1`
Batch audit multiple Turtle patch features	`task restore-batch <N>`	Restorer	Sequentially checks next N topics from queue
Compile, verify 0 errors, commit, and push ready packages	`task 1`	Single-Writer	MSVC compiles Release binaries, commits, and pushes
Re-export master printable offline reference PDF	`task pdf`	Documentation	Renders `docs/COMMAND_REFERENCE.pdf` via Microsoft Edge

7. Exhaustive Command Runbook (Each Command in a Dedicated Paragraph)

7.1. Inspect Live Pipeline Status (`task status`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" status
```

Goal: Query the health and real-time backlog of the entire porting and restoration pipeline.

When to Use: Run this before beginning any work session, or after a batch run to verify that all queues have been properly processed and drained.

Under the Hood: Queries `tools/queue/` for pending candidates (`01_candidates`), ready-to-build packages (`02_ready_to_build`), unbuilt diff patches (`staging_patches`), completed commits (`03_completed`), and rejected candidates (`04_rejected`). It also queries `tortoise-wow` to display the active Git HEAD commit hash and subject.

Output / Next Step: Displays a clean ASCII status box in your console. If packages exist in `02_ready_to_build/`, run `task 1` to process them.

7.2. Evaluate and Stage a Single Upstream Bugfix (`task port <sha>`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port 84f1bbccd
```

Goal: Investigate a specific VMaNGOS donor commit, check whether it applies cleanly to Turtle-WoW, and stage it for compilation without modifying the active code tree.

When to Use: When you have selected a specific donor commit from `docs/ROADMAP.md` or `CRUCIAL_COMMITS_QUEUE.csv` and want to review the code adaptation before building.

Under the Hood: Executes Tasks 2, 3, and 4 in sequence. Task 4 checks whether the donor files exist in `tortoise-wow`. Task 2 mines 22,155 forum threads to ensure Turtle didn't intentionally diverge. Task 3 checks for database migrations. If `git apply --check` passes cleanly, it stages `PORT-XXXX.json` in `02_ready_to_build/`. If Turtle context has diverged (e.g. `inGurubashiArena`), it generates an AI dossier in `tools/queue/ai_dossiers/<sha>.md` and flags the package as `AWAITING_AI_ADAPTATION`.

Output / Next Step: Package is staged in `tools/queue/02_ready_to_build/`. Inspect the package or diff, then execute `task 1` to compile and push.

7.3. Autonomous End-to-End Port & Build Gate (`task port <sha> -AutoBuild`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port 84f1bbccd -AutoBuild
```

Goal: Fully automate the backporting cycle from candidate analysis all the way to live GitHub push in a single command.

When to Use: When you want immediate, hands-off porting of a clean donor bugfix without intermediate manual steps.

Under the Hood: First runs the full staging checks of `task port <sha>`. If the patch applies cleanly, it immediately invokes Agent 1 (`task 1`). Agent 1 applies the patch, verifies safety invariants (`MAX_RACES = 11`, `sTWDebuff`, no progressive SQL columns), compiles `mangosd.exe` and `realmd.exe` via MSVC 2022 Release (requiring Exit Code 0), commits code to git with upstream attribution, pushes to `extended main`, and moves the manifest to `03_completed/`.

Output / Next Step: Commit is live on GitHub! Note: Do **not** run this command concurrently with another `-AutoBuild` in another terminal.

7.4. Batch Evaluation & Staging (`task port-batch <N> [-Tier <1-5>]`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port-batch 10
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port-batch 10 -Tier 1
```

Goal: Sequentially audit the next N unported candidates from the curated crucial queue and stage all viable ones for review.

When to Use: When preparing a batch of candidate fixes for an engineering sprint, or when focusing strictly on a specific severity tier (e.g. `-Tier 1` for crashes and memory leaks).

Under the Hood: Iterates through `CRUCIAL_COMMITS_QUEUE.csv`, skipping already-processed commits. For each candidate, runs the AI context assembler, forum check, and patch validation. Viable packages are deposited in `02_ready_to_build/`. Diverged packages receive AI dossiers. Incompatible fixes are moved to `04_rejected/`.

Output / Next Step: Check `task status` to view staged packages. When satisfied, execute `task 1` to compile and push the entire batch sequentially.

7.5. Hands-Off Batch Backport & Push (`task port-batch <N> -AutoBuild`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" port-batch 10 -AutoBuild
```

Goal: Automatically process, compile, and push multiple viable upstream fixes without any human intervention.

When to Use: Overnight runs or extended autonomous porting sessions where you want maximum progress across clean bugfixes.

Under the Hood: Combines the batch iterator with the single-writer build gate. For each candidate in the batch, if viable and clean, Agent 1 is invoked to compile and push before advancing to the next candidate. If a build fails or an invariant is violated, Agent 1 cleanly rolls back (`git checkout .`) and safely skips to the next candidate without corrupting the working tree.

Output / Next Step: All cleanly ported fixes are pushed directly to remote GitHub and documented in local history ledgers.

7.6. Database Entity Scalping & Differential Analysis (`task scalp <type> <id/name> -Diff`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" scalp item 19019 -Diff
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" scalp creature "Onyxia" -Diff
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" scalp spell 20925 -Diff
```

Goal: Perform deep field-by-field comparisons of items, creatures, or spells between historical vanilla baselines (`brotalnia/database` 143MB dump or VMaNGOS `mangos.sql`) and the local Turtle-WoW base (`tortoise-wow/sql/base/world.sql`).

When to Use: When a player reports an item or spell behaving incorrectly, when restoring vanilla stats, or when diagnosing whether Turtle intentionally rebalanced a mechanic.

Under the Hood: Parses the schema of both databases, extracts the matching row by ID or name, aligns all columns side-by-side, strips progressive columns (`patch`, `build`), and highlights Turtle-exclusive custom columns (`is_custom_turtle_item`). Reports identical vs. differing field counts.

Output / Next Step: Terminal displays a high-contrast side-by-side comparison table. Use `-ExportSql` if you want to generate a migration.

7.7. Exporting Clean Database Migrations (`task scalp <type> <id> -ExportSql`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" scalp item 19019 -ExportSql
```

Goal: Generate a production-ready, sanitized `REPLACE INTO` SQL migration script for any entity extracted from historical databases.

When to Use: When you want to update or restore an item, creature, or spell in Turtle-WoW's world database without risking progressive column contamination (`ERROR 1054: Unknown column 'patch'`).

Under the Hood: Extracts the entity row from the historical reference dump, strips all progressive columns, maps remaining columns strictly to Turtle-WoW's base schema, escapes column identifiers with proper backticks, and outputs a clean SQL script.

Output / Next Step: Script is saved to `tools/queue/staging_sql/<table_name>_<id>_<name>_sanitized.sql`. Move this script to `tortoise-wow/sql/database_updates/world/` and audit with `Audit-DatabaseMigrations.ps1`.

7.8. Online Database Oracle & 1.18.1 Client Parity (`task 6 <id/query>`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" 6 19019
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" 6 19019 -OpenBrowser
```

Goal: Cross-reference any item, NPC, spell, or quest with the official Turtle Database Viewer (`https://xian55.github.io/tortoise-db-viewer/`) and inspect rendered 3D models and tooltips.

When to Use: Whenever you need to verify how an entity is officially rendered in the live 1.18.1 client before accepting a donor database modification.

Under the Hood: Analyzes the query, checks local base SQL for matching entries, and constructs direct deep links into the official SQLite WASM client database viewer. With `-OpenBrowser`, launches Microsoft Edge or your default browser directly into the 3D model and tooltip view.

Output / Next Step: Console shows deep links and local cross-reference results. Browser opens the official 3D interactive viewer.

7.9. Tracking Live Official CDN Database Deltas (`task 6 changelog`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" 6 changelog
```

Goal: Monitor real-time database modifications, new spawns, and stat adjustments published to the official Turtle-WoW database CDN.

When to Use: Run periodically to detect if the upstream Turtle-WoW team has updated item balance, creature spawns, or quest rewards.

Under the Hood: Connects to `raw.githubusercontent.com/xian55/tortoise-db-viewer/cdn-dev/data/changelog.json` and fetches the latest changelog delta array directly.

Output / Next Step: Console outputs the list of recently modified entities, new spawns, and deleted templates.

7.10. Native Core Specification Restorer (`task restore <topic>`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" restore "Holy Strike"
```

Goal: Audit the leaked server core against official staff patch notes in the 22,155-thread forum archive to identify missing custom Turtle-WoW mechanics.

When to Use: When investigating class balance changes, custom hybrid talents, racial abilities, or features mentioned in Turtle patch notes (e.g. 1.15.0, 1.16.1, 1.17.2).

Under the Hood: First checks `tools/queue/restoration_history.json`. If previously verified, returns a cache hit in <0.05 seconds with zero double-checking. If un-audited, searches the forum archive for staff threads by `Torta [Turtle WoW Team]`, scans `tortoise-wow` C++ source and SQL for existing implementations, reports discrepancies, and logs the result permanently to `docs/CORE_RESTORATION_LEDGER.md`.

Output / Next Step: Reports feature status (`PARITY_VERIFIED` or `MISSING_IN_CORE`). If missing, re-run with `-StageTemplate`.

7.11. Templating a Core Restoration Package (`task restore <topic> -StageTemplate`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" restore "Holy Strike" -StageTemplate
```

Goal: Automatically scaffold a ready-to-implement restoration package manifest for a missing Turtle specification.

When to Use: When `task restore <topic>` detects a missing mechanic and you are ready to implement the C++ code patch.

Under the Hood: Runs the forum audit, extracts staff patch text, and writes a package manifest `CORE-XXXX.json` into `tools/queue/02_ready_to_build/`. Sets status to `READY_FOR_BUILD` and points to a dedicated staging patch file `staging_patches/CORE-XXXX-<topic>.patch`.

Output / Next Step: Write or paste the adapted C++ logic into the staged patch file, then execute `task 1` to compile and push.

7.12. Batch Core Restoration Audit (`task restore-batch <N>`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" restore-batch 5
```

Goal: Sequentially audit the next N un-audited Turtle patch topics from the curated priority queue (e.g. Holy Strike, Moonfury, Blood Frenzy, Trueshot Aura).

When to Use: When conducting broad core-parity sweeps across Turtle patch notes without manually typing every topic.

Under the Hood: Reads the priority topic queue, queries the persistent cache in `restoration_history.json`, skips topics that already achieved verified parity, and performs deep forum/codebase audits on remaining topics.

Output / Next Step: Console outputs sequential parity verdicts and updates `docs/CORE_RESTORATION_LEDGER.md`.

7.13. Single-Writer Builder & Committer Gate (`task 1`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" 1
```

Goal: Execute the official single-writer build gate, compiling all staged packages in `02_ready_to_build/` via MSVC 2022 Release, committing code to Git, and pushing to remote GitHub.

When to Use: Run this whenever one or more packages have been staged via `task port`, `task port-batch`, `task scalp`, or `task restore -StageTemplate`.

Under the Hood: Acquires the exclusive single-writer lock on `tortoise-wow`. Inspects `tools/queue/02_ready_to_build/` in sequential ID order (`PORT-XXXX` or `CORE-XXXX`). Safely skips any package marked `AWAITING_AI_ADAPTATION`. For ready packages: applies the `.patch` diff, verifies safety invariants (`Verify-TurtleCompatibility.ps1`), compiles `mangosd.exe` and `realmd.exe` via CMake (`--config Release`), asserts Exit Code 0, generates an atomic git commit with upstream attribution, pushes to `extended main`, advances the package to `03_completed/`, and updates all local ledgers.

Output / Next Step: Repository is updated, binaries are built, and changes are live on GitHub.

7.14. Deep Forum Archive Investigation (`task 2 <sha/topic>`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" 2 84f1bbccd
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" 2 "Holy Strike"
```

Goal: Search the 22,155 historical Turtle-WoW official forum threads (2018–2026) for developer hotfix logs, player bug reports, or patch notes relating to a specific commit or mechanic.

When to Use: When triaging an ambiguous vanilla bugfix to verify whether Turtle developers intentionally altered the formula or designed a custom mechanic.

Under the Hood: Calls `Search-ForumArchive.ps1` to perform regex and keyword scans across thread titles and bodies. Ranks matches by relevance, developer authorship (`Torta`), and patch category.

Output / Next Step: Outputs ranked matching threads to console and writes triage evaluation to `tools/queue/01_candidates/<sha>.json`.

7.15. AI Semantic Context Assembly (`task ai-audit <sha>`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" ai-audit 84f1bbccd
```

Goal: Assemble comprehensive semantic context for a donor commit, including the donor diff, matching target files in `tortoise-wow`, surrounding C++ lines, and forum intelligence.

When to Use: When a patch fails naive `git apply` due to Turtle additions (e.g. `inGurubashiArena` or debuff streaming hooks) and you need the full context to author an adapted patch.

Under the Hood: Invokes `Invoke-AiAudit.ps1`. Extracts modified functions, searches `tortoise-wow/src/` for target symbols, captures 30 lines of surrounding code context, cross-references forum lore, and compiles an AI dossier.

Output / Next Step: Saves dossier to `tools/queue/ai_dossiers/<sha>.md`. Use the dossier to author an adapted patch in `tools/queue/staging_patches/<sha>.patch`.

7.16. Master Documentation & Offline PDF Exporter (`task pdf`)

```
& "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" pdf
```

Goal: Recompile and export the complete command reference and architecture guide into a polished, printable PDF document.

When to Use: Run this after updating command documentation, adding new tools, or whenever you need an updated offline reference.

Under the Hood: Executes `Export-CommandReferencePdf.ps1`. Parses `docs/COMMAND_REFERENCE.md`, builds clean HTML styling with print media queries (`@media print`), and invokes headless Microsoft Edge (`msedge.exe --headless --print-to-pdf`) to generate `docs/COMMAND_REFERENCE.pdf`.

Output / Next Step: Generates `docs/COMMAND_REFERENCE.pdf` (typically ~270 KB) ready for offline viewing or printing.